TASINABILIR SAGLIK MONITORU

STM32G071RB + MAX30100 + MLX90614 + ST7789

Proje Dokumani

Proje Hakkinda

Bu proje, STM32G071RB mikrodenetleyicisi kullanilarak gelistirilmis tasinabilir bir saglik monitoru sistemidir. Sistem; MAX30100 sensoru ile nabiz ve kan oksijen saturasyonu (SpO2) olcumu, MLX90614 kizilotesi sensoru ile temas gerektirmeden vucut sicakligi olcumu ve ST7789 240x240 IPS ekran uzerinde gercek zamanli grafik gosterimi yapmaktadir. Nabiz 100 BPM degerini astugunda aktif buzzer ile alarm verilmektedir.

Donanim Baglantilari

ST7789 Ekran (SPI1)

Ekran Pini	STM32 Pini	Aciklama
GND	GND	Toprak
VCC	3V3	Guc (3.3V)
SCK	PA1 (SPI1_SCK)	SPI Saat
SDA	PA7 (SPI1_MOSI)	SPI Veri
RES	PA9 (TFT_RES)	Reset
DC	PC7 (TFT_DC)	Data/Komut secici
BLK	PB0 (TFT_BLK)	Arka isik kontrolu

MAX30100 Nabiz ve SpO2 Sensoru (I2C1)

Sensor Pini	STM32 Pini	Aciklama
GND	GND	Toprak
VIN	3V3	Guc (3.3V)
SDA	PA10 (I2C1_SDA)	I2C Veri - 4.7k pull-up gerekli
SCL	PB6 (I2C1_SCL)	I2C Saat - 4.7k pull-up gerekli
INT	Bagli degil	Kesme (kullanilmiyor)

MLX90614 Sicaklik Sensoru (I2C1 - Ayni Hat)

Sensor Pini	STM32 Pini	Aciklama
GND	GND	Toprak
VIN	3V3	Guc (3.3V)
SDA	PA10 (I2C1_SDA)	MAX30100 ile ayni hat
SCL	PB6 (I2C1_SCL)	MAX30100 ile ayni hat

Aktif Buzzer (Alarm)

Buzzer Pini	STM32 Pini	Aciklama
+	PB1 (BUZZER)	GPIO Output - Alarm cikisi
-	GND	Toprak

NOT: SDA ve SCL hatlarina 3V3 ile GND arasina 4.7k ohm pull-up direnc baglanmalidir. Her iki I2C sensoru (MAX30100 ve MLX90614) ayni SDA/SCL hattini paylasir.

CubeMX Ayarlari

SPI1 Ayarlari (ST7789 Ekran)

Parametre	Deger
Mode	Full-Duplex Master
Frame Format	Motorola
Data Size	8 Bits
First Bit	MSB First
Clock Polarity (CPOL)	High
Clock Phase (CPHA)	2 Edge (Mode 3)
NSS	Software
Prescaler	16 (4 MHz @ 64MHz)
NSSP Mode	Disabled
CRC Calculation	Disabled

I2C1 Ayarlari (MAX30100 + MLX90614)

Parametre	Deger
Mode	I2C
Speed Mode	Standard Mode (100 kHz)
Clock Speed	100000 Hz
Timing	0x10B17DB5
Addressing Mode	7-bit
Analog Filter	Enabled
Digital Filter	0
SCL Pin	PB6 (AF6 - I2C1_SCL)
SDA Pin	PA10 (AF6 - I2C1_SDA)

Sistem Saati (Clock) Ayarlari

Parametre	Deger
Osilatör	HSI (16 MHz dahili)
PLL Durumu	Acik (ON)

PLL Kaynagi	HSI
PLLM	1
PLLN	8
PLLR	2
SYSCLK	64 MHz (PLL)
AHB Bolücü	1 (64 MHz)
APB1 Bolücü	1 (64 MHz)
Flash Latency	2 Wait States

GPIO Ayarlari

Pin	Yon	Fonksiyon	Hiz
PA1	AF	SPI1_SCK	Yuksek
PA7	AF	SPI1_MOSI	Yuksek
PA9	Output PP	TFT_RES	Dusuk
PA10	AF OD	I2C1_SDA	Dusuk
PB0	Output PP	TFT_BLK	Dusuk
PB1	Output PP	BUZZER	Dusuk
PB6	AF OD	I2C1_SCL	Dusuk
PC7	Output PP	TFT_DC	Dusuk
PA5	Output PP	LED_GREEN	Yuksek
PA2	AF	USART2_TX	-
PA3	AF	USART2_RX	-

Kaynak Kod (main.c)

```
/* main.c - Tasinabilir Saglik Monitoru */
#include "main.h"
#include
#include
#include "st7789.h"
#include "fonts.h"
#include "max30100.h"

I2C_HandleTypeDef hi2c1;
SPI_HandleTypeDef hspi1;
UART_HandleTypeDef huart2;

/* ---- TANIMLAMALAR ---- */
#define MLX90614_ADDR  0xB4
#define MLX90614_TOBJ1 0x07
#define GRAPH_W        240
#define GRAPH_H        138
#define GRAPH_MID      69
#define PANEL_Y        141
#define WAVE_LEN       240
#define BUZZER_PORT    GPIOB
#define BUZZER_PIN     GPIO_PIN_1
#define BPM_HIGH       100

int16_t  wave_buf[WAVE_LEN];
uint16_t wave_idx = 0;
MAX30100_HandleTypeDef hmax30100;

/* ---- PRINTF YONLENDIRME ---- */
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2;, (uint8_t *)ptr, len, HAL_MAX_DELAY);
    return len;
}

/* ---- SICAKLIK OKUMA ---- */
float Read_Temp(void) {
    uint8_t buf[3];
    if (HAL_I2C_Mem_Read(&hi2c1;, MLX90614_ADDR, MLX90614_TOBJ1,
                         1, buf, 3, 100) == HAL_OK) {
        uint16_t raw = (buf[1] << 8) | buf[0];
        return (raw * 0.02f) - 273.15f;
    }
    return -99.0f;
}

/* ---- GRAFIK SUTUN CIZIMI ---- */
void Draw_Column(uint16_t col, int16_t y_new, int16_t y_prev) {
    if (y_new < 2)            y_new  = 2;
    if (y_new > GRAPH_H - 2)  y_new  = GRAPH_H - 2;
    if (y_prev < 2)           y_prev = 2;
    if (y_prev > GRAPH_H - 2) y_prev = GRAPH_H - 2;
    int16_t ymin = y_prev < y_new ? y_prev : y_new;
    int16_t ymax = y_prev > y_new ? y_prev : y_new;
    for (int16_t y = 0; y < GRAPH_H; y++) {
        uint16_t color;
        if (y >= ymin && y <= ymax)  color = GREEN;
        else if (y == GRAPH_MID)     color = 0x0300;
        else if (y % 35 == 0)        color = 0x0180;
        else if (col % 60 == 0)      color = 0x0180;
        else                         color = BLACK;
        ST7789_DrawPixel(col, y, color);
    }
}

/* ---- SAYI GUNCELLEME ---- */
void Update_Number(uint16_t x, uint16_t y, const char *str, uint16_t color) {
    for (uint16_t row = y; row < y + 26; row++)
        for (uint16_t col = x; col < x + 75; col++)
            ST7789_DrawPixel(col, row, BLACK);
    ST7789_WriteString(x, y, str, Font_16x26, color, BLACK);
}

/* ---- STATIK ARAYUZ ---- */
void Draw_Static_UI(void) {
    ST7789_Fill_Color(BLACK);
```

```
    for (uint16_t x = 0; x < 240; x++) {
        ST7789_DrawPixel(x, 139, 0x8410);
        ST7789_DrawPixel(x, 140, 0x4228);
    }
    for (uint16_t y = PANEL_Y; y < 240; y++) {
        ST7789_DrawPixel(80, y, 0x4228);
        ST7789_DrawPixel(160, y, 0x4228);
    }
    ST7789_WriteString(8,   PANEL_Y+3, "NABIZ", Font_7x10, 0x8410, BLACK);
    ST7789_WriteString(88,  PANEL_Y+3, "SpO2",  Font_7x10, 0x8410, BLACK);
    ST7789_WriteString(168, PANEL_Y+3, "ATES",  Font_7x10, 0x8410, BLACK);
    ST7789_WriteString(24,  228, "bpm", Font_7x10, 0x4228, BLACK);
    ST7789_WriteString(108, 228, "%",   Font_7x10, 0x4228, BLACK);
    ST7789_WriteString(183, 228, "C",   Font_7x10, 0x4228, BLACK);
    Update_Number(2,   PANEL_Y+16, "---", 0x4228);
    Update_Number(82,  PANEL_Y+16, "--",  0x4228);
    Update_Number(162, PANEL_Y+16, "--",  0x4228);
    for (uint16_t col = 0; col < GRAPH_W; col++)
        Draw_Column(col, GRAPH_MID, GRAPH_MID);
}

/* ---- BUZZER ALARM ---- */
void Buzzer_Alarm(void) {
    for (uint8_t i = 0; i < 3; i++) {
        HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_SET);
        HAL_Delay(100);
        HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_RESET);
        HAL_Delay(100);
    }
}

/* ---- ANA PROGRAM ---- */
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_USART2_UART_Init();
    MX_SPI1_Init();

    HAL_GPIO_WritePin(BLK_PORT, BLK_PIN, GPIO_PIN_SET);
    HAL_Delay(50);
    ST7789_Init();
    Draw_Static_UI();
    MAX30100_Init(&hmax30100;, &hi2c1;);

    for (uint16_t i = 0; i < WAVE_LEN; i++) wave_buf[i] = GRAPH_MID;

    float ir_dc=0, red_dc=0, ir_ac=0, red_ac=0;
    float ir_ac_max=0, red_ac_max=0, bpm=0, spo2=0;
    uint8_t beat_detected = 0;
    uint32_t last_beat_time = HAL_GetTick();
    const float alpha = 0.90f;
    float bpm_sum=0, spo2_sum=0, temp=0;
    uint16_t sample_count = 0;
    uint32_t window_start=HAL_GetTick(), last_temp_time=HAL_GetTick();
    int16_t last_pixel_y = GRAPH_MID;
    temp = Read_Temp();

    while (1) {
        if (MAX30100_Read_Raw(&hmax30100;) != HAL_OK) { HAL_Delay(10); continue; }

        ir_dc  = (alpha*ir_dc)  + ((1.0f-alpha)*hmax30100.IR_Raw);
        ir_ac  = hmax30100.IR_Raw - ir_dc;
        red_dc = (alpha*red_dc) + ((1.0f-alpha)*hmax30100.RED_Raw);
        red_ac = hmax30100.RED_Raw - red_dc;

        if (ir_ac > 5000.0f || ir_ac < -5000.0f) { ir_dc = hmax30100.IR_Raw; ir_ac = 0; }

        float ac_c = ir_ac;
        if (ac_c >  800.0f) ac_c =  800.0f;
        if (ac_c < -800.0f) ac_c = -800.0f;
        int16_t py = (int16_t)(GRAPH_MID - (ac_c*(GRAPH_MID-5)/800.0f));
        if (py < 2) py = 2;  if (py > GRAPH_H-2) py = GRAPH_H-2;

        Draw_Column(wave_idx % GRAPH_W, py, last_pixel_y);
        last_pixel_y = py;
        wave_buf[wave_idx] = py;
```

```
        wave_idx = (wave_idx+1) % WAVE_LEN;

        if (ir_ac > ir_ac_max)  ir_ac_max  = ir_ac;
        if (red_ac > red_ac_max) red_ac_max = red_ac;

        if (ir_ac > 0.5f && beat_detected == 0) {
            beat_detected = 1;
            uint32_t now = HAL_GetTick();
            uint32_t dt  = now - last_beat_time;
            last_beat_time = now;
            if (dt > 300 && dt < 1500) {
                float nb = 60000.0f/dt;
                bpm = (bpm < 1.0f) ? nb : (bpm*0.7f + nb*0.3f);
                if (ir_dc>0 && red_dc>0 && ir_ac_max>0) {
                    float r = (red_ac_max/red_dc)/(ir_ac_max/ir_dc);
                    float s = 104.0f - (17.0f*r);
                    if (s>100) s=100;  if (s<70) s=70;
                    spo2 = (spo2 < 1.0f) ? s : (spo2*0.7f + s*0.3f);
                }
                if (bpm>40 && bpm<160 && spo2>70)
                    { bpm_sum+=bpm; spo2_sum+=spo2; sample_count++; }
            }
            ir_ac_max = 0;  red_ac_max = 0;
        }
        if (ir_ac < -1.0f) beat_detected = 0;

        if (HAL_GetTick() - window_start >= 1000) {
            if (sample_count > 0) { bpm=bpm_sum/sample_count; spo2=spo2_sum/sample_count; }
            char buf[12];
            uint16_t bc = (bpm > BPM_HIGH) ? RED : GREEN;
            if (bpm > 1.0f) {
                snprintf(buf,12,"%3d",(int)bpm);  Update_Number(2, PANEL_Y+16, buf, bc);
                snprintf(buf,12,"%3d",(int)spo2); Update_Number(82,PANEL_Y+16, buf, YELLOW);
            }
            if (temp > -50.0f) {
                snprintf(buf,12,"%2d",(int)temp); Update_Number(162,PANEL_Y+16, buf, RED);
            }
            if (bpm > BPM_HIGH && bpm < 200) Buzzer_Alarm();
            else HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_RESET);
            bpm_sum=0; spo2_sum=0; sample_count=0;
            window_start = HAL_GetTick();
        }
        if (HAL_GetTick() - last_temp_time > 3000) {
            float t = Read_Temp();
            if (t > -50.0f) temp = t;
            last_temp_time = HAL_GetTick();
        }
        HAL_Delay(20);
    }
}
```